Contents

Plateforme Bancaire Microservices avec Assistant IA **1**
Sommaire 1
Architecture 1
Services Métier 2
Assistant Conversationnel 3
Services Techniques 3
Tests 3
Démarrage 4
Modèles de Données 5
Flux d'Exécution 6
Couches Architecture 6
Règles Métier Validées 6
Technologies 7
Structure du Projet 7
Testing 7
Statut du Projet 8
Références 8

Plateforme Bancaire Microservices avec Assistant IA

Sommaire

- Architecture
- Services
- Tests
- Démarrage

Architecture

Diagram Architecture

```
    API Clients (REST, GraphQL, Web)


         Gateway    (8080)



Acc     Tra
```

```
PostgreSQL
```

Composants: - Account-Service (8081) - Gestion des comptes - Transaction-Service (8082) - Gestion des transactions - Assistant-Service (8083) - Agent IA conversationnel - Gateway (8080) - Routage - Eureka (8761) - Service Discovery - Config-Service (8888) - Configuration centralisée

Services Métier

Account-Service

REST Endpoints: - POST /api/accounts - Créer un compte - GET /api/accounts/{id} - Obtenir un compte - GET /api/accounts - Lister les comptes - PUT /api/accounts/{id} - Modifier

Interfaces: REST, GraphQL, MCP

Test:

```
POST /api/accounts
{
  "customerReference": "client-001",
  "currency": "EUR",
  "balance": 1000
}
```

Response: UUID créé, statut ACTIVE

Transaction-Service

REST Endpoints: - POST /api/transactions - Créer une transaction - GET /api/transactions - Lister les transactions - GET /api/transactions/{id} - Obtenir une transaction - POST /api/transactions/transfers - Virement

Interfaces: REST, GraphQL, MCP

Règles métier: - Montant positif 1000 EUR - Compte doit être ACTIVE - Devise = devise du compte - Solde mis à jour après transaction

Communication Inter-Services

OpenFeign + Resilience4J: - Circuit Breaker protège les appels - Retry automatique (3 tentatives) - Pas d'interruption du service

Assistant Conversationnel

Spring AI + MCP Tools

Outils disponibles: 1. get_account - Récupère les détails d'un compte 2. list_transactions - Liste les transactions 3. create_transaction - Crée une transaction 4. transfer - Effectue un virement

Exemple:

```
Utilisateur: "Affiche les transactions du compte [id]"
Agent: Appelle list_transactions via MCP
Résultat: Transactions listées et formatées
```

Flux: Utilisateur → LLM → Sélection outils → Exécution MCP → Microservices → Réponse

Services Techniques

Spring Cloud Gateway

- Routage centralisé
- Load balancing
- CORS et rate limiting

Eureka Discovery Service

- Services s'enregistrent au démarrage
- Découverte dynamique
- Détection défaillances

Config-Service

- Configuration centralisée
- Paramètres: bank.transaction.max-amount=1000
- Profils: dev, test, prod

Tests

Résultats

Test	Résultat
Création compte	Succès
Création transaction	Succès
Validation montant	Succès
Validation devise	Succès
Compte inactif	Rejet correct
Chatbot listing	Succès
H2 Database	OK

Test	Résultat
Eureka Discovery	OK
Gateway Routing	OK
MCP Integration	OK

Screenshots

Account Service

- Création de compte via REST (Postman)
- Réponse avec UUID, statut ACTIVE

Transaction Service

- Swagger UI documentation
- Création et validation des transactions
- H2 Database Console
- H2 Database Tables

Technical Services

- Gateway configuration
- Eureka Server - Service registry
- Config-Service management

Assistant IA

- MCP Agent requests
- Chatbot responses
- Transaction listing via MCP

Démarrage

Prérequis

- Java 21+
- Maven 3.9+
- Docker & Docker Compose
- PostgreSQL (ou Docker)

Installation

```
# 1. Cloner le repository
git clone <repo>
cd elhimar-mohammed-test-adria-pfe-2027

# 2. Builder les services
```

```
mvn clean package

# 3. Démarrer l'infrastructure
docker-compose up -d

# 4. Vérifier les services
curl http://localhost:8761  # Eureka
curl http://localhost:8080  # Gateway
```

URLs des Services

Service	Port	URL
Gateway	8080	http://localhost:8080
Account-Service	8081	http://localhost:8081
Transaction-Service	8082	http://localhost:8082
Assistant-Service	8083	http://localhost:8083
Eureka	8761	http://localhost:8761
Config-Service	8888	http://localhost:8888
Swagger	8082	http://localhost:8082/swagger-ui.html
H2 Console	8081	http://localhost:8081/h2-console

Modèles de Données

Account

```
id: UUID
customerReference: String
status: ACTIVE | BLOCKED
currency: String (EUR, USD, MAD)
balance: double
```

Transaction

```
id: UUID
accountId: UUID
amount: BigDecimal
currency: String
merchantCategory: String
country: String
createdAt: Instant
```

TransactionEvent

```
eventId: UUID
eventType: "TransactionCreated"
transactionId: UUID
```

```
accountId: UUID
amount: BigDecimal
currency: String
occurredAt: Instant
```

Flux d'Exécution

Création de Transaction

1. User envoie requête POST /api/transactions
2. Transaction-Service valide les règles métier
3. OpenFeign appelle Account-Service pour vérifier le compte
4. Resilience4J gère les défaillances
5. Solde du compte est mis à jour
6. Transaction est persistée
7. Événement est publié dans Kafka
8. Réponse retournée à l'utilisateur

Agent Conversationnel

1. Utilisateur envoie message via /api/chat
2. Spring AI envoie au LLM
3. LLM analyse et sélectionne les outils
4. MCP exécute les outils (get_account, list_transactions, etc.)
5. Microservices traitent les requêtes
6. Résultats retournés au LLM
7. LLM formate la réponse finale
8. Réponse retournée à l'utilisateur

Couches Architecture

Chaque microservice respecte:

```
Domain (Entités métier)
  ↓
Repository (Accès données)
  ↓
Service (Logique métier)
  ↓
Controllers (REST, GraphQL, MCP)
```

Les trois contrôleurs appellent la même couche service.

Règles Métier Validées

1. Montant strictement positif
2. Montant plafond configuré (1000)
3. Compte existe et est ACTIVE

4. Devise transaction = devise compte
5. Solde consulté et mis à jour
6. Virements inter-comptes supportés

Technologies

Framework: Spring Boot, Spring Cloud, Spring AI **Communication:** REST, GraphQL, MCP, OpenFeign, Kafka **Database:** PostgreSQL, H2 (tests) **Container:** Docker, Docker Compose **CI/CD:** GitHub Actions **Autres:** Resilience4J (circuit breaker), Eureka (service discovery)

Structure du Projet

```
/account-service
    src/main/java
        domain/          # Entités
        repository/      # Accès données
        service/         # Logique métier
        web/
            rest/        # Contrôleurs REST
            graphql/     # Contrôleurs GraphQL
        mcp/             # Outils MCP

/transaction-service
    (structure identique)

/assistant-service
    (agent conversationnel)

/gateway-service
    (Spring Cloud Gateway)

/discovery-service
    (Eureka Server)

/config-service
    (Configuration centralisée)
```

Testing

Test une Requête

```bash
# Account-Service
curl -X POST http://localhost:8081/api/accounts \
  -H "Content-Type: application/json" \
  -d '{"customerReference":"test","currency":"EUR","balance":1000}'
```

```
# Transaction-Service
curl -X POST http://localhost:8082/api/transactions \
  -H "Content-Type: application/json" \
  -d '{"accountId":"[uuid]","amount":100,"currency":"EUR","merchantCategory":"FOOD","country

# Chatbot
curl -X POST http://localhost:8083/api/chat \
  -H "Content-Type: application/json" \
  -d '{"message":"Affiche les transactions du compte [uuid]"}'
```

Statut du Projet

Complété: - Part B - Architecture & Design - Part C - Business Microservices - Part D - Technical Services - Part E - AI Chatbot - OpenFeign + Resilience4J - REST, GraphQL, MCP - Tests et validation

Infrastructure: - Docker support - Docker Compose - CI/CD pipeline ready

Références

- Spring Cloud: https://spring.io/projects/spring-cloud
- Spring AI: https://spring.io/projects/spring-ai
- OpenFeign: https://github.com/OpenFeign/feign
- Resilience4J: https://resilience4j.readme.io/
- Docker: https://docs.docker.com/

Auteur: Mohammed EL HIMAR
Date: 2026-10-30
Status: Complet et testé